



Artificial Intelligence (CIS4049-N)

Week 6 – Reinforcement Learning: Implementation & Project Ideas

Module Leader: Dr. Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk

Today's plan

- **Reinforcement Learning (RL) – Implementation & projects ideas**
 - Recap – Basic Concepts
 - Project 1: Solving the taxi environment with Q-Learning
 - Project 2: Self-driving cab with SARSA (State-Action-Reward-State-Action)
 - LLMs as Reinforcement Learning agents
 - Conclusions and project Ideas

Recap – Basic Concepts

Basic concepts: RL, Reward, Policy

- **What is RL?** An intelligent agent needs to learn, through trial and error, how to take actions inside and environment in order to maximize a cumulative reward.
- **Reward** - a scalar feedback signal that indicates how well agent is doing at step t and it defines the goal.
- **Policy** - A policy is a learned mapping from states to actions. Solving a reinforcement learning problem means **finding the best possible policy**. Policies are either **deterministic** (map each state to one action):

$$\pi(state) = \text{action}$$

- or **stochastic** (map each state to a probability distribution over all possible actions).

$$\pi(state) = (\text{prob action 1, prob action 2, ... prob action N})$$

Several methods to compute this optimal policy: **policy optimization methods**.

Basic concepts: Value, Value function, Bellmann



- Sometimes, depending on the problem, instead of directly trying to find the optimal policy, we can try to find the **value function** associated with that optimal policy.
- The **value** is a number associated with each state s of the environment that estimates how good it is for the agent to be in state s . It is the **cumulative reward** the agent collects when starting at state s and choosing actions according to policy π . A **value function** is a learned mapping from states to values:

$$v_{\pi}(s) = \text{cumulative reward when the agent starts at state } s \text{ and follows policy } \pi$$

- Value functions can also map pairs of (action, state) to values. In this case, they are called **q-value** functions:

$$q_{\pi}(s, a) = \text{cumulative reward when the agent start at state } s \text{ takes action } a \text{ and follows policy } \pi \text{ thereafter}$$

- The optimal value function (or **q-value** function) satisfies a mathematical equation, called the **Bellman equation**:

$$q_{*}(s, a) = \mathbb{E}[r + \gamma \max_{a'} q_{*}(s', a')]$$

Q- learning

$$q_*(s, a) = \mathbb{E}[r + \gamma \max_{a'} q_*(s', a')]$$

- This equation is useful because it can be transformed into an iterative procedure to find the optimal value function.
- *Why are value functions useful?* Because you can infer an optimal policy from an optimal q-value function.
- How? The **optimal policy** is the one where at each state s the agent chooses the action a that **maximizes the q-value function**.
- So, you can jump from **optimal policies** to **optimal q-functions**, and vice versa.
- There are several RL algorithms that focus on finding optimal q-value functions. These are called **Q-learning** methods.

RL Algorithms & OpenAI Gym/Gymnasium



- There are **several different RL algorithms**. Some try to directly **find optimal policies**, others **q-value functions**, and others **both at the same time**.
- No one-size-fits-all when it comes to RL algorithms. You need to experiment with a few of them each time you solve an RL problem and see what works for your case. The best practice to learn these algorithms is to implement them and gain an insight into what works best in each situation.
- **Reinforcement learning agents are VERY data-hungry**. Solution: using **simulated environments**. Writing the engine that simulates the environment usually requires more work than solving the RL problem. Also, changes between different engine implementations can render comparisons between algorithms meaningless.
- This is why **OpenAI** released the [Gym toolkit](#) back in 2016. OpenAI's gym offers a **standardized API** for a **collection of environments for different problems**, including the classic Atari games, robotic arms or landing on the Moon. OpenAI Gym also defines a **standard API to build environments**, allowing third parties (like you) to create and make your environments available to others.
- **Gymnasium (<https://gymnasium.farama.org/>)** is a maintained fork of OpenAI's Gym library. The Gymnasium interface is simple, pythonic, and capable of representing general RL problems, and has a [compatibility wrapper](#) for old Gym environments (*"import gymnasium as gym"*)

RL in Python

```
import random

def train(n_episodes: int):
    """
    Pseudo-code of a Reinforcement Learning agent training loop
    """

    # python object that wraps all environment logic. Typically you will
    # be using OpenAI gym here.
    env = load_env()

    # python object that wraps all agent policy (or value function)
    # parameters, and action generation methods.
    agent = get_rl_agent()

    for episode in range(0, n_episodes):

        # random start of the environmnet
        state = env.reset()

        # epsilon is parameter that controls the exploitation-exploration trade-off.
        # it is good practice to set a decaying value for epsilon
        epsilon = get_epsilon(episode)

        done = False
        while not done:
```

```
while not done:

    if random.uniform(0, 1) < epsilon:
        # Explore action space
        action = env.action_space.sample()
    else:
        # Exploit learned values (or policy)
        action = agent.get_best_action(state)

    # environment transitions to next state and maybe rewards the agent.
    next_state, reward, done, info = env.step(action)

    # adjust agent parameters. We will see how later in the course.
    agent.update_parameters(state, action, reward, next_state)

    state = next_state
```

ϵ - value between 0 and 1 that represents the probability the agent chooses a random action instead of what she thinks is the best one. Key parameter to ensure our agent explores the environment enough, before drawing definite conclusions on what is the best action to take in each state. This trade-off between exploring new strategies vs sticking to already known ones is called the **exploration-exploitation problem**.

Summing-up

- Every **RL problem** has an **agent** (or agents), **environment**, **actions**, **states** and **rewards**.
- The agent sequentially takes actions with the goal of maximizing total rewards. For that she needs to find the **optimal policy**.
- Value functions are useful as they give us an alternative path to find the optimal policy.
- In practice, you need to try different RL algorithms for your problem and see what works best.
- RL agents need a lot of training data to learn. OpenAI gym/Gymnasium is a great tool to re-use and create your environments.
- **Exploration vs exploitation** is necessary when training RL agents, to ensure the agent does not get stuck in local optima.

Project 1: Solving the taxi environment with Q-Learning

Taxi environment – Problem setting

- The goal of the **Taxi Environment** is simple and straightforward: *the taxi should pick up passengers and drop them at their destination on a small parking lot, driving the shortest path possible.*
- Step-to-step implementation of the environment, how to learn a policy using tabular Q-learning and visualize the learned behaviour in animations.



Taxi environment – State space

- An **MDP** states contains the information needed to (i) determine actions, (ii) compute the action reward and (iii) compute the transition to the next state.
- The most obvious state property is the **location** of the taxi; the $5*5$ grid gives us 25 options. Furthermore, a passenger waiting for pickup can be waiting at 1 or 4 points (labelled Y, R, G, B) or they can be in the taxi, giving us $(4+1)$ options. Finally, the destination can also be Y, R, G or B. We denote the state by the following vector:

```
State = [x_pos_taxi, y_pos_taxi, pos_passenger, dest_passenger]
```

- Although a very simple problem setting, we thus already deal with **$5*5*(4+1)*4=500$ states!** Scale the problem to thousands of locations, and it is easy to see how quickly the size of MDPs explodes (***curse of dimensionality***).
- **Note.** Reachable states < 500 (a passenger will never have the same pickup point and destination). Due to modelling complexities, we typically focus on the full state space, representing the worst-case scenario.

Taxi environment – Action space

- The state space may be bigger than you'd expect at first glance, but the **action space** contains only **six** decisions:
 - Down
 - Up
 - Left
 - Right
 - Drop passenger
 - Pick up passenger
- In case the action is not allowed (e.g., trying to drive through a wall, attempting to pick up a customer when there is none at the present location), the agent will remain still.
- The **action_mask** (found in **env.action_mask(state)**) informs us which actions are feasible, but this feasibility check is not automatically enforced.

Taxi environment – Reward function

- Three types of **reward** can be distinguished in this environment:
 - **Move: -1.** Every move comes with a **small penalty**, to encourage following the shortest path from origin to destination.
 - **Failed drop-off: -10.** A passenger will naturally be unhappy when dropped at the wrong location, so a **large penalty** is appropriate.
 - **Successful drop-off: 20.** To encourage the desired behaviour, a **large bonus** is incurred.
- Designing rewards in a way that encourages desired behaviour is known as ***reward shaping***. For instance, we might simply end the game without a reward upon successful drop-off (thus ending the stream of costs), but a clear reward signal often eases learning considerably.

Taxi environment – Transition function

- Unlike most realistic environments, the taxi environment exhibits **no randomness**. This feature considerably **simplifies learning**, as the next state is dictated solely by our action. In case an infeasible action is selected — such as trying to drive through a wall or picking up a non-existent passenger — the agent simply remains in place.
- In an abstract sense, we could denote the **transition function** as follows:

```
new_state = f(state, action)
```

- An explicit definition would consider all grid moves, (in)feasible drop-offs and (in)feasible pickups, which is not necessarily hard but requires some focus.
- The game ends upon successfully dropping off a passenger at their destination, denoting the **terminating state**.

Inizialization

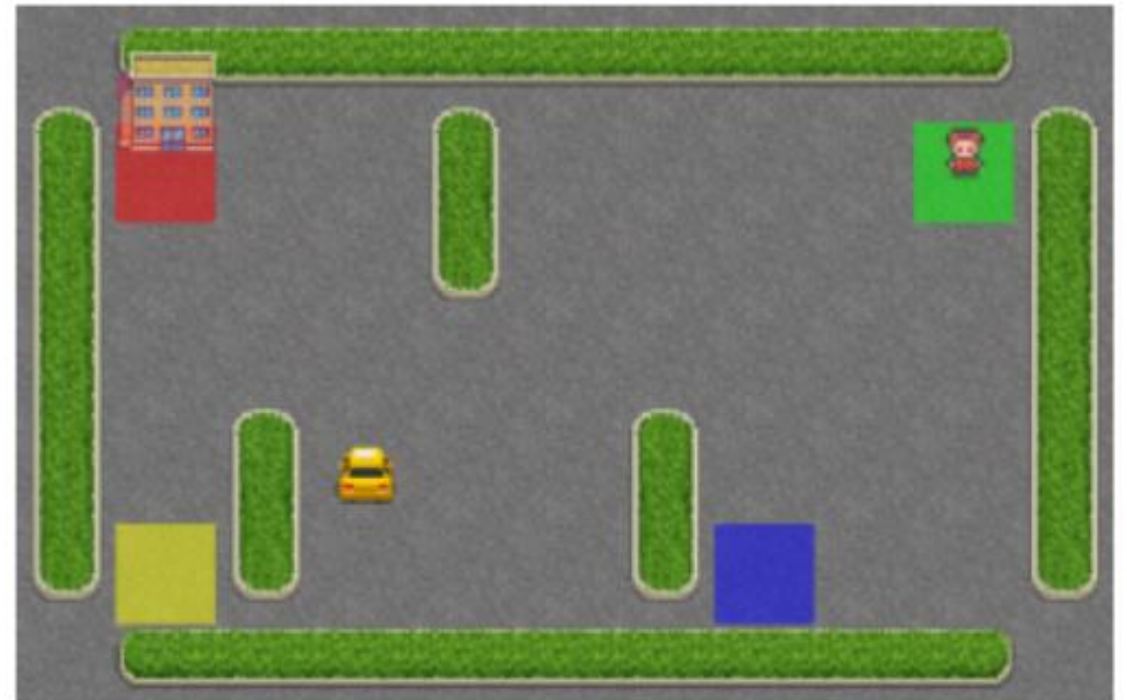
- First, we **install the necessary libraries** and subsequently **import them**. Obviously, we need to install the gym environment. Other than that, we just require some visual stuff and your typical data science libraries

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
import random
from IPython.display import clear_output
from time import sleep
from matplotlib import animation
```


Validate the environment

- Assuming all imports went correctly, it is time for a sanity check. Let's **create** and **render** a **Taxi-v3 environment**.
- We should see an action between 0 and 5, a state between 0 and 499, and an action mask to block infeasible actions.
- Most importantly, we should have rendered a still frame to affirm we are truly up and running!

```
State space: Discrete(500)  
Action space: Discrete(6)  
State: 424  
Action: 1  
Action mask: [0 1 1 0 0 0]  
Reward: -1
```



Validate the environment

- Finally, let's set up **two functions**: one for the **console output** and **animation**, one for storing GIFs.
- Note that the animation is simply a repeated plot of the RGB frame. It is also very time-consuming, and therefore not recommended to run during training.

```
def run_animation(experience_buffer):  
    """Function to run animation"""  
    time_lag = 0.05 # Delay (in s) between frames  
    for experience in experience_buffer:  
        # Plot frame  
        clear_output(wait=True)  
        plt.imshow(experience['frame'])  
        plt.axis('off')  
        plt.show()  
  
        # Print console output  
        print(f"Episode: {experience['episode']}/{experience_buffer[-1]['episode']}")  
        print(f"Epoch: {experience['epoch']}/{experience_buffer[-1]['epoch']}")  
        print(f"State: {experience['state']}")  
        print(f"Action: {experience['action']}")  
        print(f"Reward: {experience['reward']}")  
        # Pause animation  
        sleep(time_lag)
```

```
def store_episode_as_gif(experience_buffer, path='./', filename='animation.gif'):  
    """Store episode as gif animation"""  
    fps = 5 # Set frame per seconds  
    dpi = 300 # Set dots per inch  
    interval = 50 # Interval between frames (in ms)  
  
    # Retrieve frames from experience buffer  
    frames = []  
    for experience in experience_buffer:  
        frames.append(experience['frame'])  
  
    # Fix frame size  
    plt.figure(figsize=(frames[0].shape[1] / dpi, frames[0].shape[0] / dpi), dpi=dpi)  
    patch = plt.imshow(frames[0])  
    plt.axis('off')  
  
    # Generate animation  
    def animate(i):  
        patch.set_data(frames[i])  
  
    anim = animation.FuncAnimation(plt.gcf(), animate, frames=len(frames), interval=interval)  
  
    # Save output as gif  
    anim.save(path + filename, writer='imagemagick', fps=fps)
```

Testing a random agent

- Having asserted that the environment works as expected, it is time **to let a random agent run wild**. This **agent takes random action at every moment in time**. This is essentially how the learning agent sets out, having no reliable Q-values to work with.
- You will see the agent trying to drive through walls, pick up passengers at deserted sports, or drop off passenger at the wrong location. It might take thousands of actions before — strictly by accident — dropping the passenger at the right location.

```
"""Simulation with random agent"""
epoch = 0
num_failed_dropoffs = 0
experience_buffer = []
cum_reward = 0

done = False

state, _ = env.reset()

while not done:
    # Sample random action
    "Action selection without action mask"
    action = env.action_space.sample()

    "Action selection with action mask"
    #action = env.action_space.sample(env.action_mask(state))

    state, reward, done, _, _ = env.step(action)
    cum_reward += reward
```

```
# Store experience in dictionary
experience_buffer.append({
    "frame": env.render(),
    "episode": 1,
    "epoch": epoch,
    "state": state,
    "action": action,
    "reward": cum_reward,
})

if reward == -10:
    num_failed_dropoffs += 1

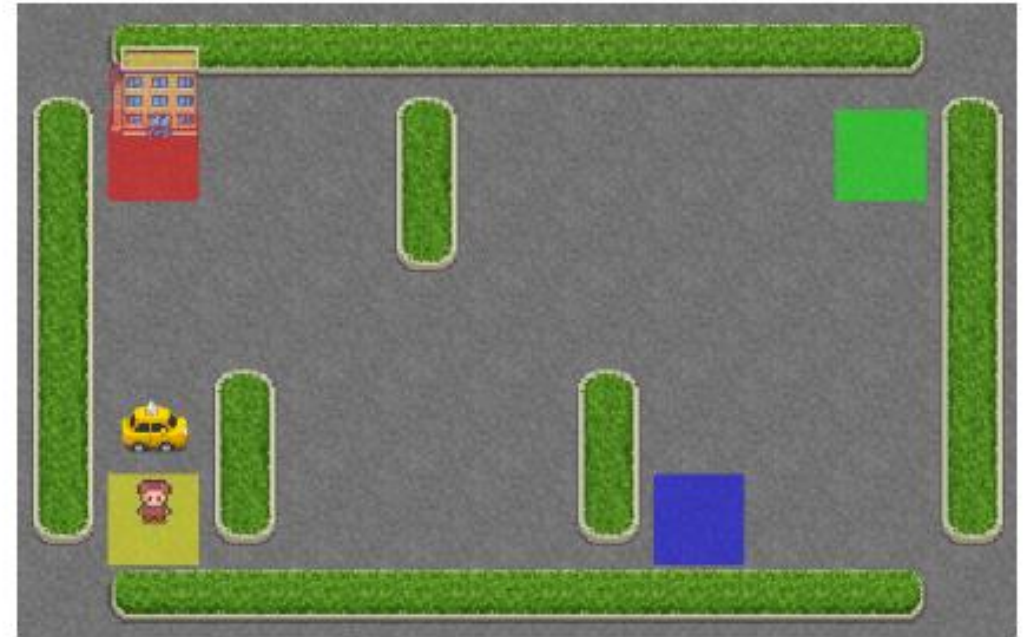
epoch += 1

# Run animation and print console output
run_animation(experience_buffer)

print("# epochs: {}".format(epoch))
print("# failed drop-offs: {}".format(num_failed_dropoffs))
```

Untrained random agent

- As you can imagine, **the random agent frustrates the passenger to no end**
- This really gives an impression how an untrained RL agent behaves, and how long it takes to get a meaningful reward signal.
- **Note.** By applying the action mask — a Boolean vector that essentially assigns a probability of 0 to selecting infeasible actions — we restrict ourselves to only sensible actions, typically substantially shortening the episode's length.



Training the agent

- Let's deploy a basic **Q-learning algorithm** with **ϵ -greedy exploration**, selecting random actions with probability ϵ and use Q-values to select actions for the remainder of the time.
- **Q-values** are updated using the following equation after making an observation.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_t)} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

- In Q-learning, we update values of state-action pairs after each observation, using **off-policy learning** (learning whilst following someone else's behaviour ("look over someone's shoulder"))
- Note that with 500 states and 6 actions, we must fill a **Q-table** of size $500 \times 6 = 3000$, with each state-action pair requiring multiple observations to learn its value with any degree of accuracy.

Training the agent

- The code looks as follows. You can tinker with the **hyper-parameters** (although in my experience this problem is fairly insensitive to them).

```
"""Training the agent"""
q_table = np.zeros([env.observation_space.n, env.action_space.n])

# Hyperparameters
alpha = 0.1 # Learning rate
gamma = 1.0 # Discount rate
epsilon = 0.1 # Exploration rate
num_episodes = 10000 # Number of episodes

# Output for plots
cum_rewards = np.zeros([num_episodes])
total_epochs = np.zeros([num_episodes])

for episode in range(1, num_episodes+1):
    # Reset environment
    state, info = env.reset()
    epoch = 0
    num_failed_dropoffs = 0
    done = False
    cum_reward = 0

    while not done:
        if random.uniform(0, 1) < epsilon:
            "Basic exploration [~0.47m]"
            action = env.action_space.sample() # Sample random action (exploration)
```

```
            "Exploration with action mask [~1.52m]"
            # action = env.action_space.sample(env.action_mask(state)) "Exploration with action mask"
        else:
            "Exploitation with action mask [~1m52s]"
            # action_mask = np.where(info["action_mask"]==1,0,1) # invert
            # masked_q_values = np.ma.array(q_table[state], mask=action_mask, dtype=np.float32)
            # action = np.ma.argmax(masked_q_values, axis=0)

            "Exploitation with random tie breaker [~1m19s]"
            # action = np.random.choice(np.flatnonzero(q_table[state] == q_table[state].max()))

            "Basic exploitation [~47s]"
            action = np.argmax(q_table[state]) # Select best known action (exploitation)

    next_state, reward, done, _, info = env.step(action)

    cum_reward += reward

    old_q_value = q_table[state, action]
    next_max = np.max(q_table[next_state])

    new_q_value = (1 - alpha) * old_q_value + alpha * (reward + gamma * next_max)

    q_table[state, action] = new_q_value
```

Training the agent

```

"""Training the agent"""
q_table = np.zeros([env.observation_space.n, env.action_space.n])

# Hyperparameters
alpha = 0.1 # Learning rate
gamma = 1.0 # Discount rate
epsilon = 0.1 # Exploration rate
num_episodes = 10000 # Number of episodes

# Output for plots
cum_rewards = np.zeros([num_episodes])
total_epochs = np.zeros([num_episodes])

for episode in range(1, num_episodes+1):
    # Reset environment
    state, info = env.reset()
    epoch = 0
    num_failed_dropoffs = 0
    done = False
    cum_reward = 0

    while not done:

        if random.uniform(0, 1) < epsilon:
            "Basic exploration [~0.47m]"
            action = env.action_space.sample() # Sample random action (exploration)
            "Exploration with action mask [~1.52m]"
            # action = env.action_space.sample(env.action_mask(state)) "Exploration with action mask"
        else:
            "Exploitation with action mask [~1m52s]"
            # action_mask = np.where(info["action_mask"]==1,0,1) # invert
            # masked_q_values = np.ma.array(q_table[state], mask=action_mask, dtype=np.float32)
            # action = np.ma.argmax(masked_q_values, axis=0)

            "Exploitation with random tie breaker [~1m19s]"
            # action = np.random.choice(np.flatnonzero(q_table[state] == q_table[state].max()))

            "Basic exploitation [~47s]"
            action = np.argmax(q_table[state]) # Select best known action (exploitation)

        next_state, reward, done, _ , info = env.step(action)

        cum_reward += reward

        old_q_value = q_table[state, action]
        next_max = np.max(q_table[next_state])

        new_q_value = (1 - alpha) * old_q_value + alpha * (reward + gamma * next_max)

        q_table[state, action] = new_q_value

```

```

        if reward == -10:
            num_failed_dropoffs += 1

        state = next_state
        epoch += 1

        total_epochs[episode-1] = epoch
        cum_rewards[episode-1] = cum_reward

    if episode % 100 == 0:
        clear_output(wait=True)
        print(f"Episode #: {episode}")

print("\n")
print("===Training completed.===\n")

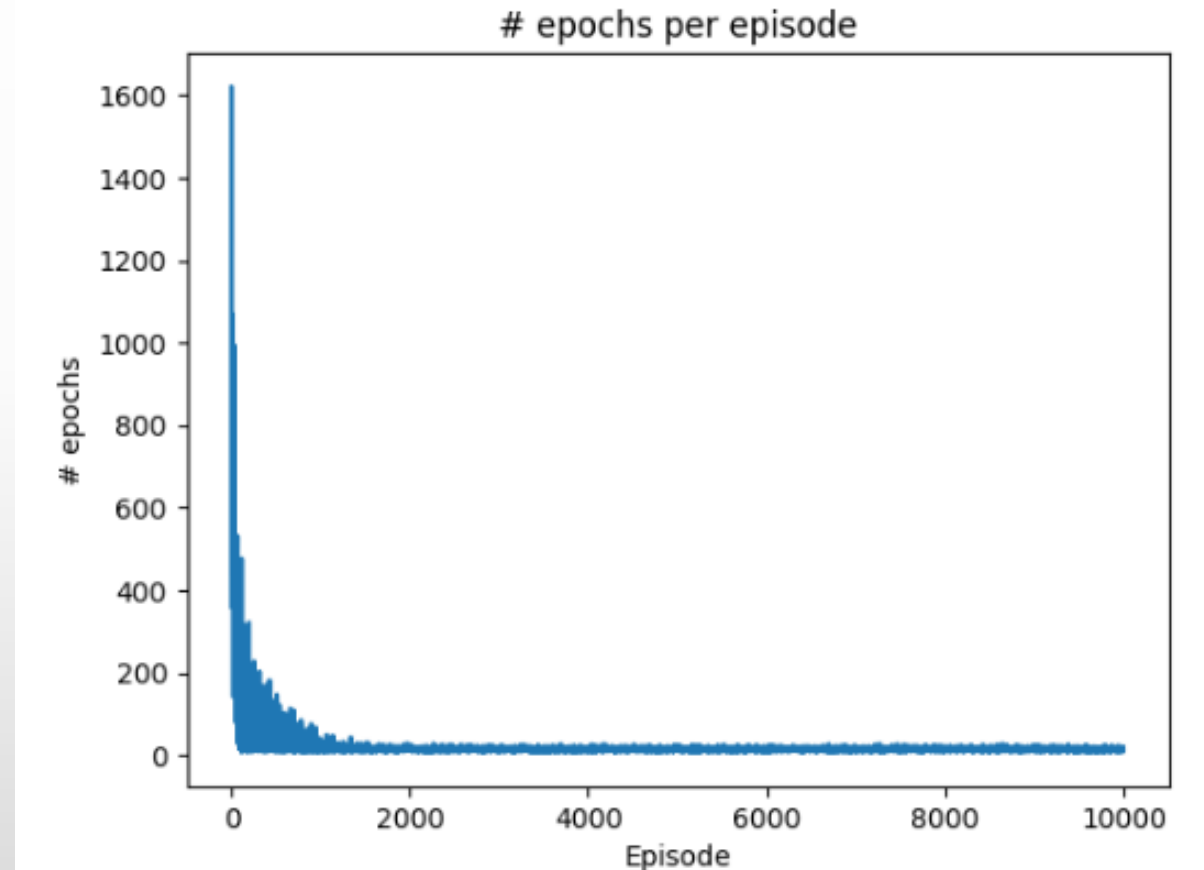
# Plot reward convergence
plt.title("Cumulative reward per episode")
plt.xlabel("Episode")
plt.ylabel("Cumulative reward")
plt.plot(cum_rewards)
plt.show()

# Plot epoch convergence
plt.title("# epochs per episode")
plt.xlabel("Episode")
plt.ylabel("# epochs")
plt.plot(total_epochs)
plt.show()

```

Convergence

- **Convergence of training** is visualized below. After 2000 episodes, it seems we have learned a pretty good and consistent policy.
- Initially, the agent requires many steps to successfully complete an episode. Ultimately, positive rewards are recognized, and the agent starts taking increasingly efficient actions, culminating in shortest path solutions.



Testing the policy

- Depending on the state we are in, we look up the corresponding Q-values in the Q-table (i.e., six values for each state, corresponding to the actions) and select the action with the **highest associated Q-value**.
- Effectively, the Q-value captures the **expected cumulative reward** associated with the action. Note that we no longer select random actions with probability ϵ — this mechanism was required strictly for learning.
- If you performed sufficiently many iterations, you should see the taxi always drive directly to the passenger, take the shortest path to the destination, and successfully drop of the passenger. Indeed, we've come a long way since driving around at random.

```
"""Test policy performance after training"""
```

```
num_epochs = 0
total_failed_deliveries = 0
num_episodes = 1
experience_buffer = []
store_gif = True

for episode in range(1, num_episodes+1):
    # Initialize experience buffer

    my_env = env.reset()
    state = my_env[0]
    epoch = 1
    num_failed_deliveries = 0
    cum_reward = 0
    done = False

    while not done:
        action = np.argmax(q_table[state])
        state, reward, done, _, _ = env.step(action)
        cum_reward += reward

        if reward == -10:
            num_failed_deliveries += 1

    # Store rendered frame in animation dictionary
    experience_buffer.append({
        'frame': env.render(),
        'episode': episode,
        'epoch': epoch,
        'state': state,
        'action': action,
        'reward': cum_reward
    })

    epoch += 1

    total_failed_deliveries += num_failed_deliveries
    num_epochs += epoch

    if store_gif:
        store_episode_as_gif(experience_buffer)

# Run animation and print output
run_animation(experience_buffer)

# Print final results
print("\n")
print(f"Test results after {num_episodes} episodes:")
print(f"Mean # epochs per episode: {num_epochs / num_episodes}")
print(f"Mean # failed drop-offs per episode: {total_failed_deliveries / num_episodes}")
```

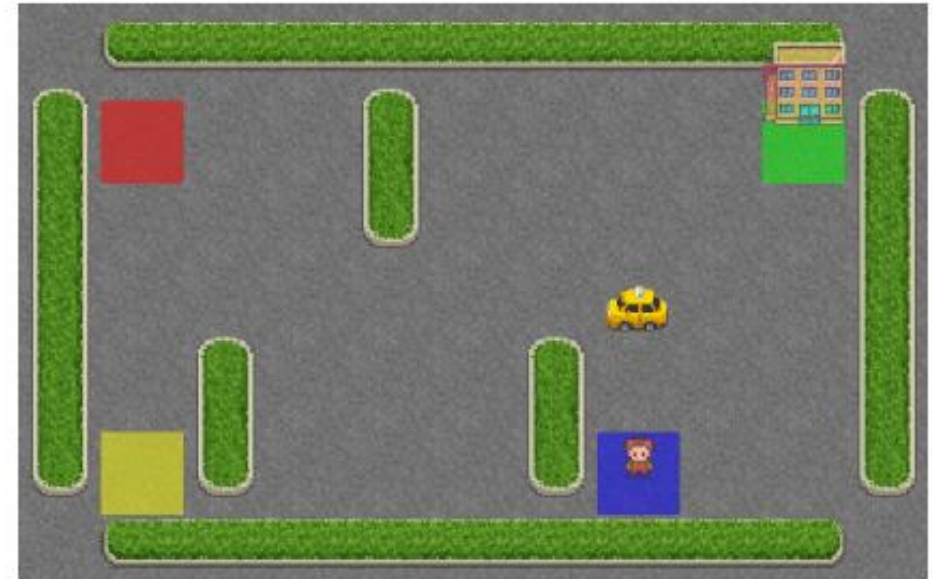
Testing the policy

- Depending on the state we are in, we look up the corresponding Q-values in the Q-table (i.e., six values for each state, corresponding to the actions) and select the action with the **highest associated Q-value**.
- Effectively, the Q-value captures the **expected cumulative reward** associated with the action. Note that we no longer select random actions with probability ϵ — this mechanism was required strictly for learning.
- If you performed sufficiently many iterations, you should see the taxi always drive directly to the passenger, take the shortest path to the destination, and successfully drop of the passenger. Indeed, we've come a long way since driving around at random.

```
"""Test policy performance after training"""
```

```
num_epochs = 0
total_failed_deliveries = 0
num_episodes = 1
experience_buffer = []
store_gif = True

for episode in range(1, num_episodes+1):
    # Initialize experience buffer
```



```
num_epochs += epoch

if store_gif:
    store_episode_as_gif(experience_buffer)

# Run animation and print output
run_animation(experience_buffer)

# Print final results
print("\n")
print(f"Test results after {num_episodes} episodes:")
print(f"Mean # epochs per episode: {num_epochs / num_episodes}")
print(f"Mean # failed drop-offs per episode: {total_failed_deliveries / num_episodes}")
```

Wrapping-up

- Although it requires many observations to learn this policy in a seemingly trivial environment, you might appreciate the result more if you realize the algorithm learned all of this without knowing anything at all about the environment it operates in!
- The **Taxi environment** is a nice one to get started with Reinforcement Learning. The problem setting is simple and intuitive yet could easily be extended towards something more realistic (think realistic urban networks, managing taxi fleets, uncertain travel times). It is computationally well-manageable, but also illustrates the computational effort required even to learn policies for trivial problem settings.
- If you are new to Reinforcement Learning, playing around with the **OpenAI Gym/Gymnasium environments** is highly recommended before moving to designing and solving your own problems.
- **Future work:** Turn this code into a module of functions that can use multiple environments and work with the hyperparameters (alpha, gamma and epsilon), optimising them by implementing a grid search to discover the best hyperparameters.

Project 2: Self-driving cab with SARSA (State-Action-Reward-State-Action)

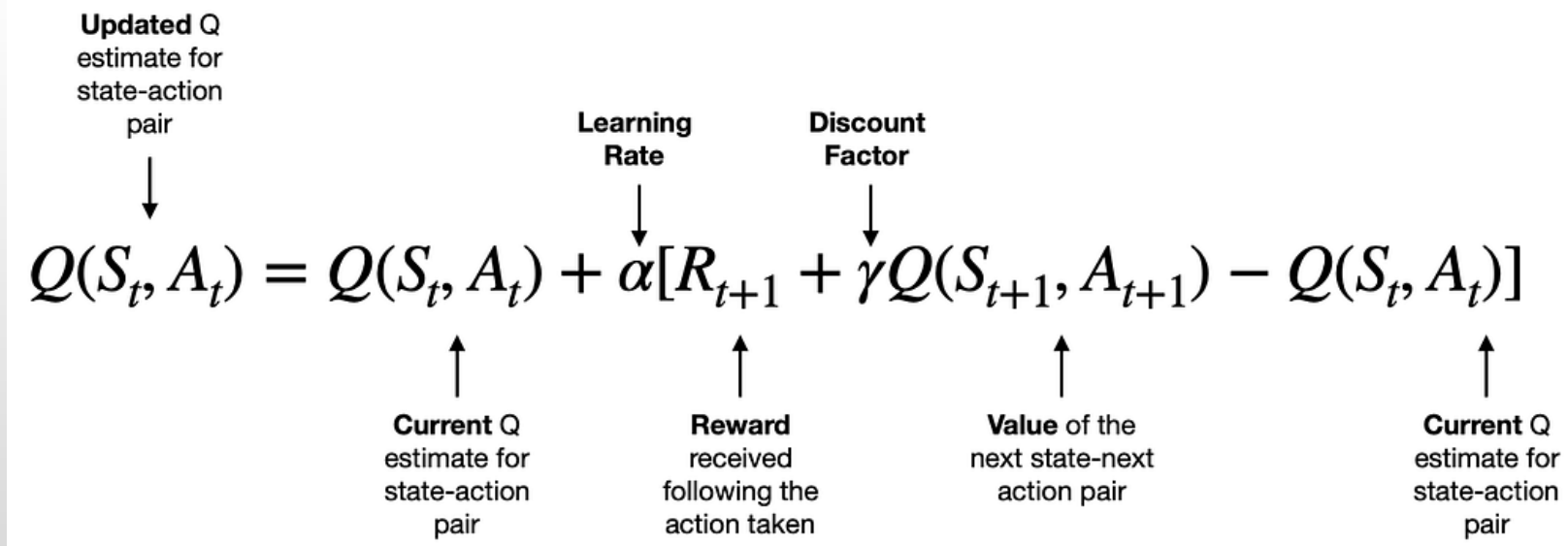
SARSA (State-Action-Reward-State-Action)

- **SARSA** is a **Value-based RL method** similar to Q-learning. Hence, it uses a **Q-table** to store values for each **state-action pair**. With value-based strategies, we train the agent **indirectly** by teaching it to identify which states (or state-action pairs) are more valuable.
- Typically, we start with all values in a **Q-table initialised to 0**, and we use **training to optimise the Q-table**. Then our agent can use information stored in a Q-table to choose the best action at each state (i.e., the action with the highest value for each state is the one selected by the agent).
- Here is an example of an empty Q-table:

		Action Space				
State Space	Q-table	Action (0)	Action (1)	...	Action (m-1)	Action (m)
	0	0	0	0	0	0
	1	0	0	0	0	0
	2	0	0	0	0	0
	...	0	0	0	0	0
	n-1	0	0	0	0	0
	n	0	0	0	0	0

SARSA algorithm

- **SARSA** is an *on-policy* algorithm, which is one of the areas **differentiating it from Q-Learning** (off-policy algorithm). *On-policy* means that during training, we use the same policy for the agent to act (*acting policy*) and to update the value function (*updating policy*). Meanwhile, with the off-policy approach, we use different policies for acting and updating.
- Now let's take a look at the SARSA algorithm itself:


$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

The diagram illustrates the SARSA update equation with labels for each term:

- Updated Q estimate for state-action pair** (points to the left $Q(S_t, A_t)$)
- Learning Rate** (points to α)
- Discount Factor** (points to γ)
- Reward received following the action taken** (points to R_{t+1})
- Value of the next state-next action pair** (points to $Q(S_{t+1}, A_{t+1})$)
- Current Q estimate for state-action pair** (points to the right $Q(S_t, A_t)$)

SARSA algorithm

The diagram shows the SARSA update equation with arrows pointing to each term to explain its meaning:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Annotations:

- Updated Q estimate for state-action pair** points to the left-hand side $Q(S_t, A_t)$.
- Current Q estimate for state-action pair** points to the first $Q(S_t, A_t)$ on the right-hand side.
- Learning Rate** points to the α term.
- Reward received following the action taken** points to the R_{t+1} term.
- Discount Factor** points to the γ term.
- Value of the next state-next action pair** points to the $Q(S_{t+1}, A_{t+1})$ term.
- Current Q estimate for state-action pair** points to the second $Q(S_t, A_t)$ on the right-hand side.

- Q is the **value function**, and the term on the left $Q(S_t, A_t)$ is the **new value** for the specific state-action pair. Note, S refers to State, and A refers to Action.
- On the right-hand side of the equation, we find the same term $Q(S_t, A_t)$, which, in this case, is the **current value** for that same state-action pair.
- To update the current value, we take the **reward** R_{t+1} following the action taken by the agent, add the **value for the next state-next action pair** $\gamma Q(S_{t+1}, A_{t+1})$ **discounted by gamma**, and subtract the **current value** $Q(S_t, A_t)$.
- So, the terms in the square brackets produce a positive, zero or negative value, which leads to either increase, no change or a decrease in the new value of $Q(S_t, A_t)$. Note that we also apply a **learning rate (α)** to control the “size” of each update.
- Since SARSA uses **Temporal Difference (TD)** approach, the algorithm will keep updating the Q-table after each step until we reach the maximum number of iterations, or the solution converges to an optimal one.

SARSA vs Q-Learning

- If we look at the equation used by the Q-Learning algorithm, we can see that the **difference lies in how it selects the value of the next state.**

- **Q-Learning takes the maximum value** for the next state based on the existing values in the Q-table. Meanwhile, **SARSA takes the value of the next state-next action pair.**

$$\begin{array}{c} \text{Updated Q estimate for state-action pair} \\ \downarrow \\ Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \boxed{\max_a Q(S_{t+1}, a)} - Q(S_t, A_t)] \\ \begin{array}{ccccccc} \uparrow & & \uparrow & & \uparrow & & \uparrow \\ \text{Current Q estimate for state-action pair} & & \text{Reward received following the action taken} & & \text{Maximum value of the next state} & & \text{Current Q estimate for state-action pair} \end{array} \end{array}$$

Q-Learning

$$\begin{array}{c} \text{Updated Q estimate for state-action pair} \\ \downarrow \\ Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \\ \begin{array}{ccccccc} \uparrow & & \uparrow & & \uparrow & & \uparrow \\ \text{Current Q estimate for state-action pair} & & \text{Reward received following the action taken} & & \text{Value of the next state-next action pair} & & \text{Current Q estimate for state-action pair} \end{array} \end{array}$$

SARSA

SARSA vs Q-Learning

$$Q^{new}(s, a) = Q^{old}(s, a) + \alpha * \underbrace{(r + \gamma * Q^{old}(s', a'))}_{\text{target}} - Q^{old}(s, a)$$

- Both SARSA and Q-learning exploit the Bellman equation to iteratively find better approximations to the optimal q-value function $Q^*(s, a)$ but they do it slightly differently. SARSA has a similar update formula but with a different target that depends also on the action a' the agent will take in the next state s' . This is the final **A** in SARSA's name. If you explore enough the state space and update your q-matrices with SARSA you will get to an **optimal policy**.
- **SARSA's** update depends on the next action a' , and hence on the current policy. As you train and the q-value (and associated policy) get updated the new policy might produce a different next action a'' for the same state s' . You cannot use past experiences (s, a, r, s', a') to improve your estimates. Instead, you use each experience once to update the q-values and then throw it away. Because of this, SARSA is called an **on-policy** method.
- In **Q-learning**, the update formula does not depend on the next action a' , but only on (s, a, r, s') . You can reuse past experiences (s, a, r, s') , collected with an old version of the policy, to improve the q-values of the current policy. Q-learning is an **off-policy** method.
- **Off-policy** methods need less experience to learn than **on-policy** methods because you can re-use past experiences several times to improve your estimates (more sample efficient). However, off-policy methods have issues converging to the optimal q-value function $Q^*(s, a)$ when the state, action spaces grow. They can be tricky and unstable.

Taxi environment – Setup and checking

```
# Setup the environment
env = gym.make(id='Taxi-v3', # choose one of the existing environments
               max_episode_steps=None, # default=None, Maximum length of an episode (TimeLimit wrapper).
               autoreset=False, # default=None, Whether to automatically reset the environment after each episode (AutoResetWrapper).
               disable_env_checker=None, # default=None, If to run the env checker
               render_mode = 'rgb_array' # The set of supported modes varies per environment. (And some third-party environments may not support rendering at all.)
               )
```

```
# Show environment description (map) as an array
```

```
print("Environment Array: ")
```

```
print(env.desc)
```

```
# Observation and action space
```

```
state_obs_space = env.observation_space # Returns state(observation) space of the environment.
```

```
action_space = env.action_space # Returns action space of the environment.
```

```
print("State(Observation) space:", state_obs_space)
```

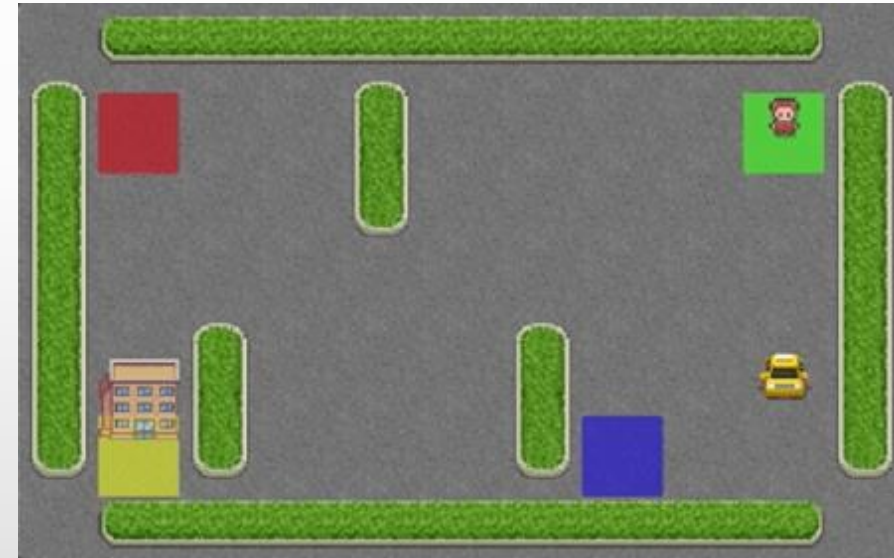
```
print("Action space:", action_space)
```

Environment Array:

```
[[b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-']
 [b'|' b'R' b':' b'|' b'|' b'|' b'|' b':' b'|' b':' b'|' b'|']
 [b'|' b'| b':' b'| b'| b'| b'| b':' b'| b':' b'| b'|']
 [b'|' b'| b':' b'| b'| b'| b'| b':' b'| b':' b'| b'|']
 [b'|' b'| b'| b'| b'| b'| b'| b'| b'| b'| b'| b'|']
 [b'|' b'Y' b'| b'| b'| b'| b'| b'| b'B' b'| b'| b'|']
 [b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-' b'+-']
```

State(Observation) space: Discrete(500)

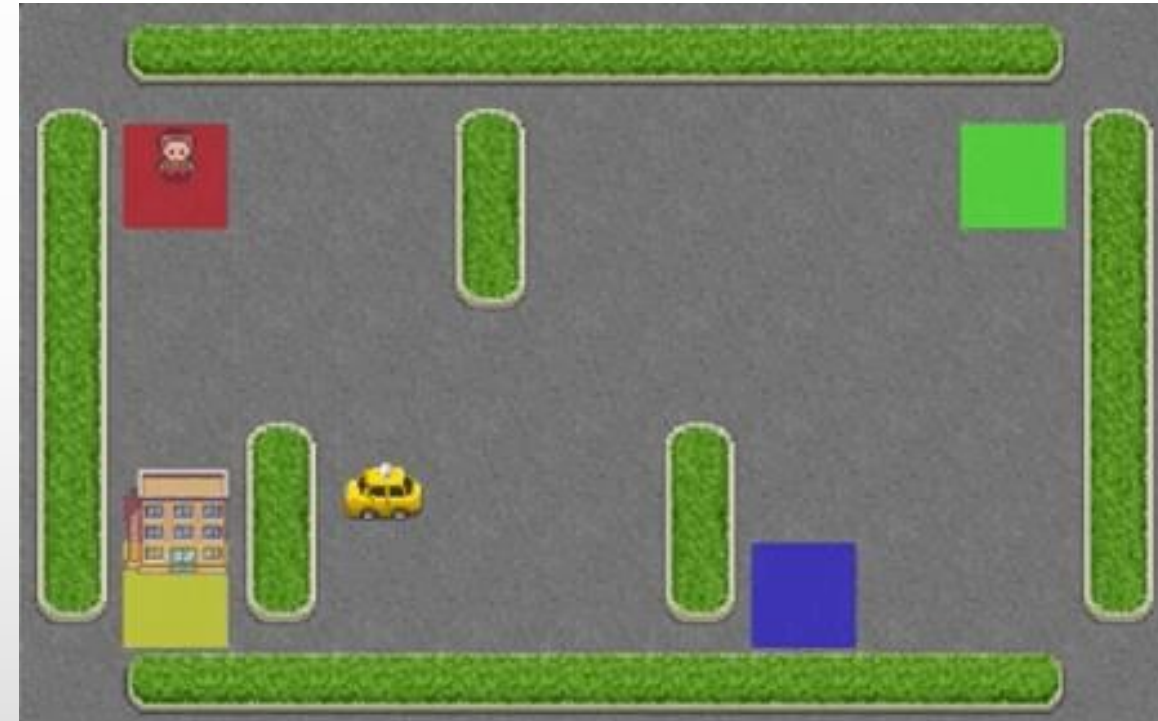
Action space: Discrete(6)



Taxi environment – Setup and checking

- The state space contains **500 discrete states**, representing all possible combinations of 25 taxi positions, 5 possible passenger locations (including the case when the passenger is in the taxi), and 4 destination locations, i.e., $25 * 5 * 4 = 500$. Meanwhile, the **actions** are:
 - 0: move south
 - 1: move north
 - 2: move east
 - 3: move west
 - 4: pick-up a passenger
 - 5: drop-off a passenger
- Before we start model training, let's make our agent (taxi car) perform random actions and render its movements around the environment.

```
# Reset environment to initial state
state, info = env.reset()
# Cycle through 30 random steps rendering and displaying the agent inside the environment each time
for _ in range(30):
```



```
display.clear_output(wait=True) # clear output before showing the next frame
# Reset environment
state, info = env.reset()

# Close environment
env.close()
```

Taxi environment – Model training

- With the setup complete, let's use **SARSA** to find **the best policy(π)** for our agent to follow in this game. We start by initialising a few parameters.
- To balance **exploration vs exploitation**, we will vary **epsilon** throughout training. We will start with $\epsilon = 1$ (pure exploration) and decay epsilon with every episode to gradually move from pure exploration to exploitation.
- Next, let's initialise the **Q-table**. As we've seen in the previous section, it will be a 500 x 6 table with rows representing states and columns representing actions. We initialise the Q-table with all 0's since we do not know how valuable each state is before we start the training.

```
# SARSA parameters
alpha = 0.1 # learning rate
gamma = 0.95 # discount factor

# Training parameters
n_episodes = 100000 # number of episodes to use for training
n_max_steps = 100 # maximum number of steps per episode

# Exploration / Exploitation parameters
start_epsilon = 1.0 # start training by selecting purely random actions
min_epsilon = 0.0 # the lowest epsilon allowed to decay to
decay_rate = 0.00001 # epsilon will gradually decay so we do less exploring and more exploiting
```

```
# Initial Q-table
# Our Q-table is a matrix of state(observation) space x action space, i.e., 500 x 6
Qtable = np.zeros((env.observation_space.n, env.action_space.n))

# Show
Qtable
```

Taxi environment – Training a

- SARSA is an **on-policy** algorithm, so we will use the same **epsilon-greedy strategy** to act and update the Q-table.
- Let's now define functions to use in training and evaluation. If you look closely at the **update_Q function**, you will see that it contains the SARSA algorithm equation defined earlier.
- Finally, let's define our **training function**.

```
def train(n_episodes, n_max_steps, start_epsilon, min_epsilon, decay_rate, Qtable):
    for episode in range(n_episodes):

        # Reset the environment at the start of each episode
        state, info = env.reset()
        t = 0
        done = False

        # Calculate epsilon value based on decay rate
        epsilon = max(min_epsilon, (start_epsilon - min_epsilon)*np.exp(-decay_rate*episode))

        # Choose an action using previously defined epsilon-greedy policy
        action = epsilon_greedy(Qtable, state, epsilon)

        for t in range(n_max_steps):

            # Perform the action in the environment, get reward and next state
            next_state, reward, done, _, info = env.step(action)

            # Choose next action
            next_action = epsilon_greedy(Qtable, next_state, epsilon)

            # Update Q-table
            Qtable = update_Q(Qtable, state, action, reward, next_state, next_action)

            # Update current state
            state = next_state
            action = next_action

            # Finish the episode when done=True, i.e., reached the goal or fallen into a hole
            if done:
                break

        # Return final Q-table
    return Qtable
```


Taxi environment – Training and Evaluation

- Now we can call the training function and see the **results**.
- Following the training, we get the Q-table as an output. However, it is difficult to say if it is optimal. Hence, we will need to do some additional evaluation. Here is a snippet of a **Q-table post-training**.

```
# Train
Qtable = train(n_episodes, n_max_steps, start_epsilon, min_epsilon, decay_rate, Qtable)

# Show Q-table
Qtable
```

```
array([[ 0.          ,  0.          ,  0.          ,  0.          ,
         0.          ,  0.          ],
       [-15.88478151, -14.78900627, -16.69581259, -14.66434947,
        -12.6181495 , -23.80156577],
       [-6.19468628, -2.88294982, -6.49649702, -2.24076811,
        -0.99650814, -13.12010172],
       ...,
       [-4.25834456,  2.30997394, -5.05986368, -3.97161199,
        -12.53096203, -15.46710745],
       [-17.96569618, -12.49725207, -16.13686029, -10.22935569,
        -24.18094778, -27.10102464],
       [ 12.22333031,  6.43016417, 12.76697657, 15.36125068,
         4.18029065,  2.6316856 ]])
```

Taxi environment – Evaluation

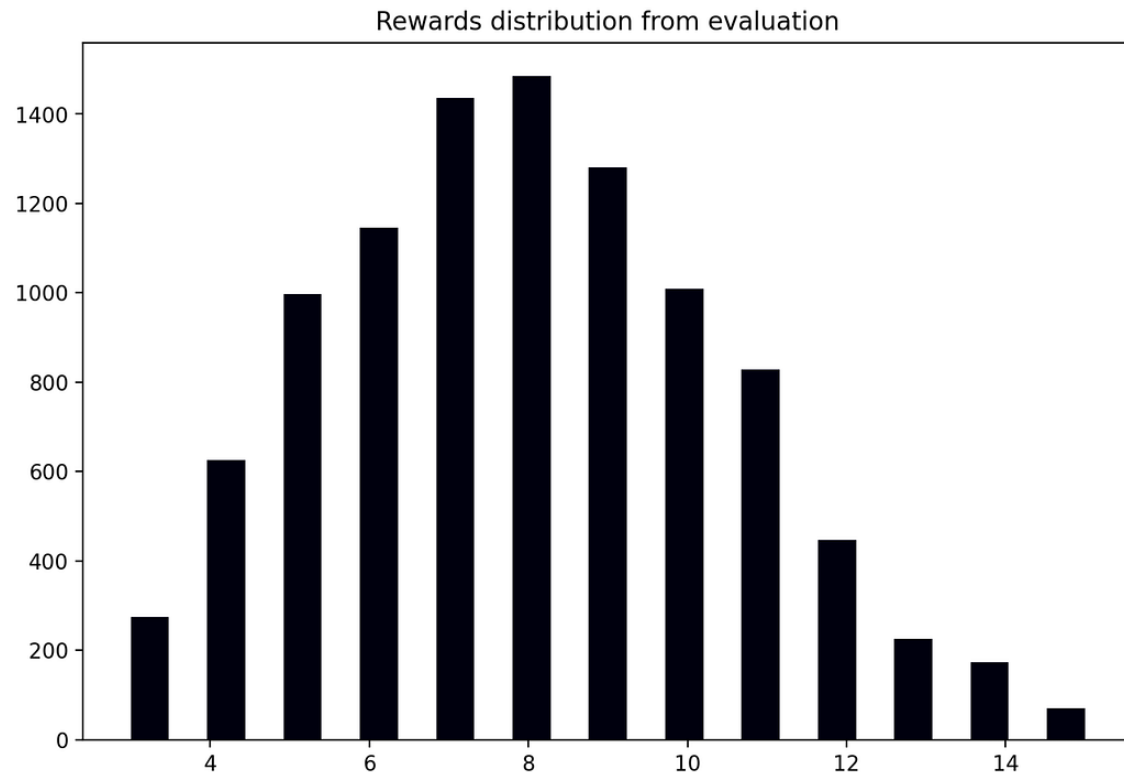
- Since it is difficult to judge the result by inspecting the Q-table, let's run **10,000** episodes to evaluate the agent's performance.
- We will use statistics such as **mean, standard deviation** and **min/max** of the rewards alongside the **distribution chart**.

```
def evaluate_agent(n_max_steps, n_eval_episodes, Qtable):  
    # Initialize an empty list to store rewards for each episode  
    episode_rewards=[]  
  
    # Evaluate for each episode  
    for episode in range(n_eval_episodes):  
  
        # Reset the environment at the start of each episode  
        state, info = env.reset()  
        t = 0  
        done = False  
        tot_episode_reward = 0  
  
        for t in range(n_max_steps):  
  
            # Use greedy policy to evaluate  
            action = eval_greedy(Qtable, state)  
  
            # Pass action into step function  
            next_state, reward, done, _, info = env.step(action)  
  
            # Sum episode rewards  
            tot_episode_reward += reward  
  
            # Update current state  
            state = next_state  
  
            # Finish the episode when done=True, i.e., reached the goal or fallen into a hole  
            if done:  
                break  
  
        episode_rewards.append(tot_episode_reward)  
  
    mean_reward = np.mean(episode_rewards)  
    std_reward = np.std(episode_rewards)  
  
    return mean_reward, std_reward, episode_rewards
```

Taxi environment – Results

- We call the above function and display the results.

Mean Reward = 7.96 +/- 2.59
Min = 3.0 and Max 15.0



```
# Evaluate
n_eval_episodes=10000
mean_reward, std_reward, episode_rewards = evaluate_agent(n_max_steps, n_eval_episodes, Qtable)

# Print evaluation results
print(f"Mean Reward = {mean_reward:.2f} +/- {std_reward:.2f}")
print(f"Min = {min(episode_rewards):.1f} and Max {max(episode_rewards):.1f}")

# Show the distribution of rewards obtained from evaluation
plt.figure(figsize=(9,6), dpi=200)
plt.title(label='Rewards distribution from evaluation', loc='center')
plt.hist(episode_rewards, bins=25, color='#00000f')
plt.show()
```


Taxi environment – Results

- The **average reward** from **10,000 episodes** is **7.96 +/- 2.59**. It indicates some variation in rewards, but that is because at the start of each episode, the locations of a taxi, a passenger, and a drop-off change. Hence, each time we require a different number of steps to complete the task, so we don't always receive the same reward (each move is minus 1 point).
- The **minimum reward** is **3**, which confirms that we always get a positive reward. Looking at the map, we can easily calculate that it takes **17 steps** (16 moves + 1 pick-up action) if we initialised the taxi and passenger at opposite corners with a drop-off location being the same corner as the taxi's original position. Hence, choosing the perfect route for a most distant scenario would result in a reward of 3 (20 - 17).
- The **maximum reward** is **15**, so let's see why we cannot expect any higher. If a passenger is initialised at the drop-off location, the episode ends immediately (i.e., not a valid scenario, so there is no hope for scoring 20). Hence, the best case we can hope for is when the passenger and taxi are initialised in the same location. If we take the two closest colour squares (Red and Yellow), the agent will use 5 moves (1 to pick up and 4 to move) to transport the passenger from the pick-up location to the drop-off location, giving us the maximum possible reward of 15 (20 - 5).
- The reward distribution chart indicates a **close-to-normal distribution of rewards**. So, while it does not prove that we have an optimal policy, it suggests that the agent is making rational moves.

Taxi environment – Results

- One way to **ensure that we have an optimal policy** would be to calculate the **minimum number of steps required for each initial state**. Then validate that the agent never takes more than the minimum number of steps (good exercise).
- Instead, let me show a **visualisation of an agent performing actions following our policy**.
- The code gives us the following visualisation, which confirms that, at **least in the two scenarios captured below, the agent has taken an optimal approach**.

```
# Reset environment to initial state
state, info = env.reset()

# Cycle through 50 steps rednering and displaying environment state each time
for _ in range(50):
```

```
    # Render and display current state of the environment
    plt.imshow(env.render()) # render current state and pass to pyplot
```



```
state, info = env.reset()
```

```
env.close()
```

LLMs as Reinforcement Learning agents

RL in Language Models

- LLMs (GPT-4, Mixtral, Llama etc.) need the ability to align with human preferences, generate meaningful context and understand language semantics.
- Typical training process through autoregressive generation and stochastic gradient descent optimization don't always lead to optimal results.
- *Deep Reinforcement Learning* (DRL) has proven to be immensely effective for improving LLMs.
- Integrating DRL helps the LLMs to dynamically adjust their behavior based on feedback. (e.g., [RLHF](#))

Reinforcement learning with Human Feedback (RLHF)



- **RLHF** is a process used to fine-tune language models by incorporating **human feedback** into their training.
- Instead of relying solely on autoregressive objective, it introduces a reward system based on human evaluations to optimize the training process.
- E.g., given some summaries generated by the model, **the goal is to maximize the reward for the summary that aligns better with human preferences**

The stock market saw a significant dip due to global events

Global events caused fluctuations resulting in losses across industries

- Human feedback or a trained reward model assigns:

$R(\text{Summary 1}) = 8.5$ (more concise and relevant)

$R(\text{Summary 2}) = 5.5$ (less focused on key points)

- These reward values are used to optimize the LLMs to **better map human preferences**

Common RLHF pipeline

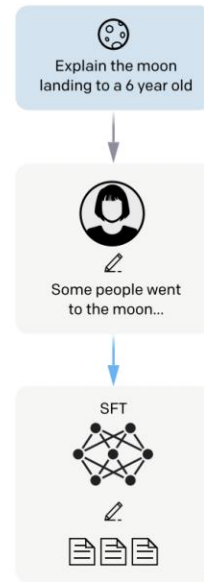
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



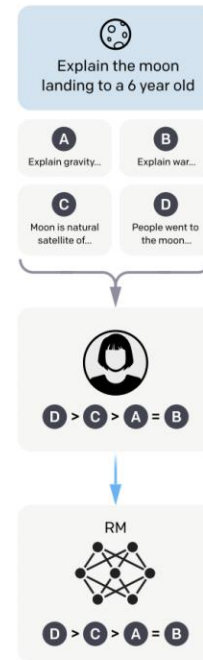
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

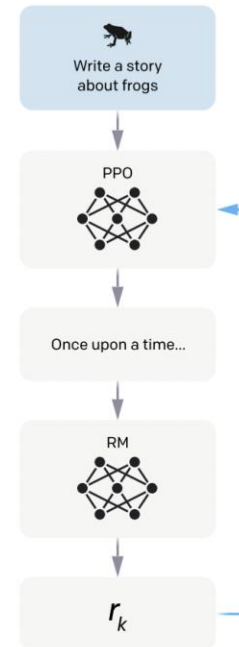
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

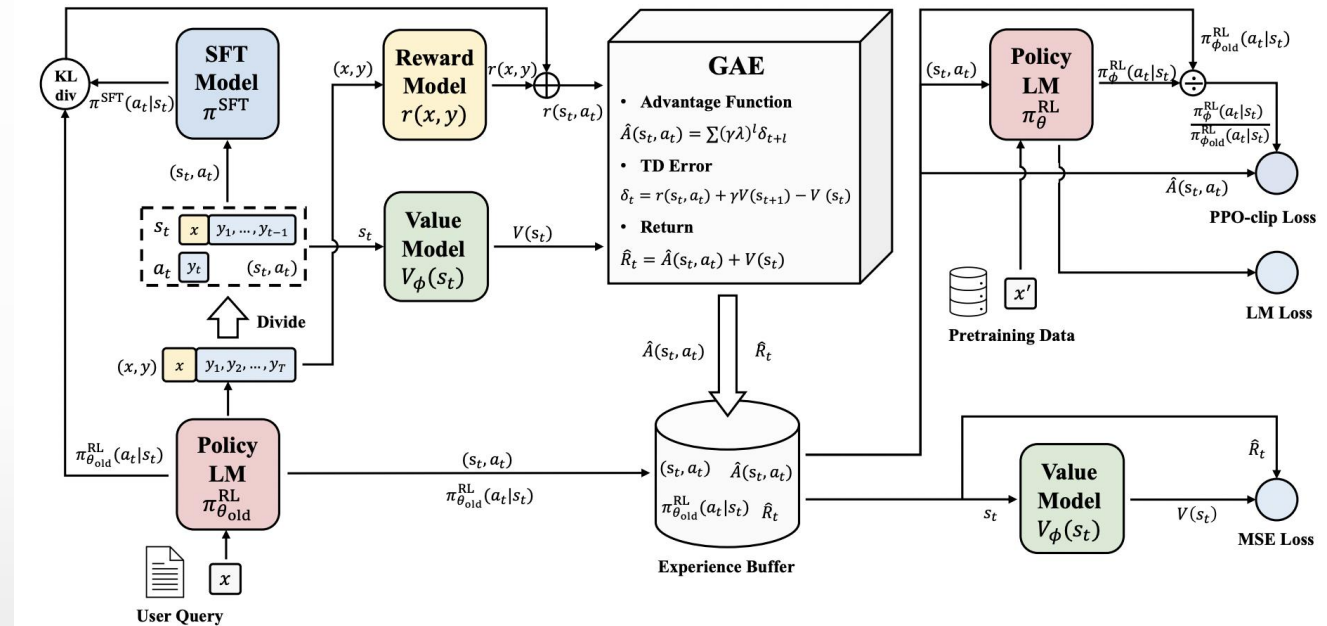
The reward is used to update the policy using PPO.



[[Stanford cs224n](#)]

RL optimization can be extremely complex

- Estimating future rewards is challenging, especially with long sequences.
- Updating the model's action selection to maximize rewards requires efficient gradient updates.
- The KL-divergence penalty adds complexity, as it requires balancing to prevent the model from drifting too far from its pre-trained state.



LLMs as Agents in RL Environments

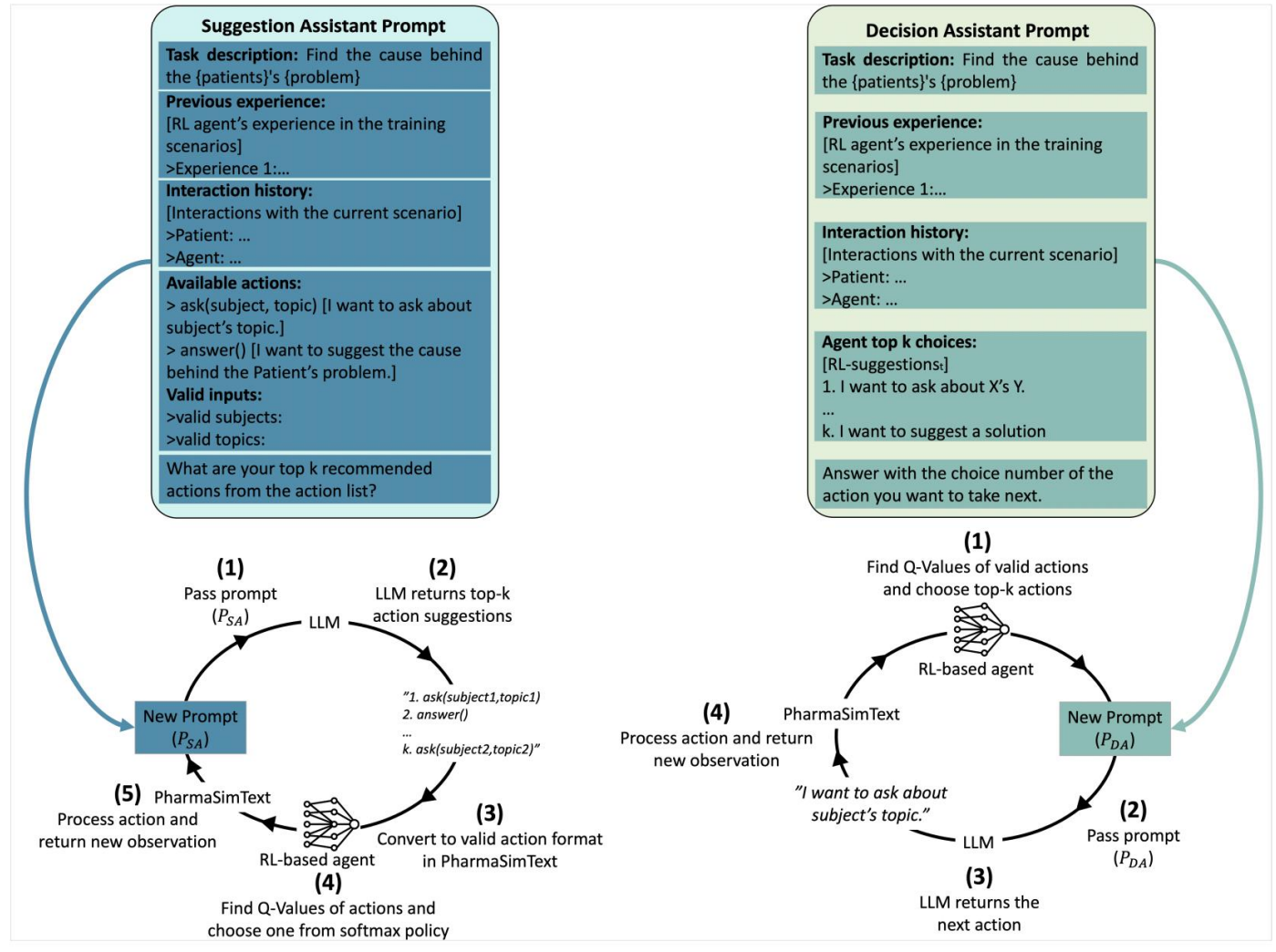
LLMs like GPT-4 and LLaMA are not only powerful in NLP tasks but also show promise as **agents** in reinforcement learning environments.

- **Why LLMs as Agents?** – LLMs come **pre-trained** with a vast amount of world knowledge and language understanding which allows them to adapt to various problem spaces without the need for extensive retraining.
- **Dynamic Problem-Solving** – They are excellent at **interpreting context** and **generating flexible responses** which makes them useful in decision-making tasks that require reasoning and strategy.
- **Zero-Shot and Few-Shot Capabilities** – LLMs can **generalize across tasks** with few or no examples, which reduces the need for extensive reward signals from the environment (i.e., less reliance on trial-and-error feedback).

Scenario I: Diagnostic conversation with a patient

- Combine the **decision-making strength** of RL with the **reasoning and natural language capabilities** of Large Language Models (LLMs)
- In Q-learning, an agent learns to map states to actions that maximize long-term rewards.
- The agent explores and receives feedback on actions, refining its strategy over time.
- **LLMs assist the RL agent by suggesting possible actions at each step:**
 - Improves the agent's ability to handle unfamiliar scenarios.
 - Helps make decisions grounded in natural language reasoning.
 - Allows agents to excel in both task completion and engaging conversations

Scenario I: Diagnostic conversation with a patient



[framework]

Scenario I: Q-Learning with LLM Integration

- The RL agent learns by interacting with the **environment**, **receiving rewards**, and updating the **Q-values** for each state-action pair to estimate future rewards.
- At each step, the agent selects actions that *maximize the expected reward* using the Q-values learned during training.
- **Maximize cumulative rewards** by exploring the best possible actions over time.
- Integration of LLM can enhance the RL agent by providing **context-aware suggestions** during inference.
- Instead of relying solely on predefined state representations, the LLM interprets natural language states and suggests multiple valid actions.

Scenario I: LLM-Assisted Q-Learning in Action

- **State Representation** – The RL agent observes the current state (e.g., infant has some disease).
- **Q-Learning Update** – Uses past experience (Q-values) to evaluate possible actions and select those maximizing long-term rewards.
- **Action Selection** – The RL agent picks an action that has the highest Q-value from a set of pre-learned options.
- The LLM provides additional suggestions based on its vast knowledge and reasoning capabilities (e.g., ask about the infant's recent diet).

Process Flow

- LLM suggests key diagnostic questions like "Has the diet changed?" or "What are the symptoms?"
- **Q-Learning Evaluation** – The RL agent evaluates these suggestions by recalculating their Q-values and selecting the one that maximizes the expected reward.
- **Action Execution** – The agent asks the chosen question, continuing the dialogue based on the optimal path for diagnosis.

Scenario II: Multi-Agent Coordination with LLMs

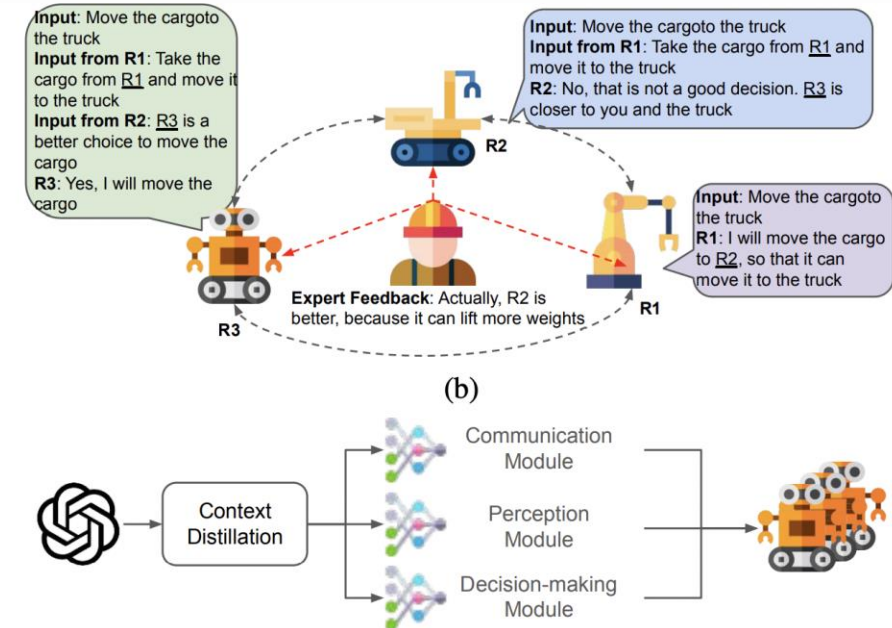
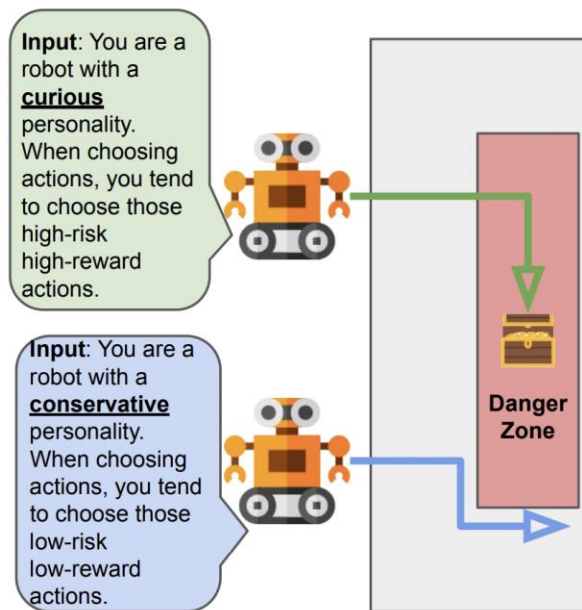
LLMs can be integrated into **multi-agent RL environments** to coordinate actions and share information to achieve a common goal (e.g., [LLMs based Multi-agent RL](#))

- **In a cooperative environment**, multiple agents equipped with LLMs are tasked with solving a shared goal (e.g., negotiating roles or deciding strategies).
- Agents can use **natural language** to communicate, exchange information about their environment.
- By using LLMs for both **decision-making** and **communication**, agents can handle dynamic and ambiguous tasks more effectively.

Scenario II: Multi-Agent Coordination with LLMs

- **Personality-Based Decision Making (Left)**
- Two agents are given distinct **personalities**, one *curious* and *risk-taking*, and the other *conservative* and *risk-averse*.
- This personality-based variation guides the agents' decisions, such as whether to:

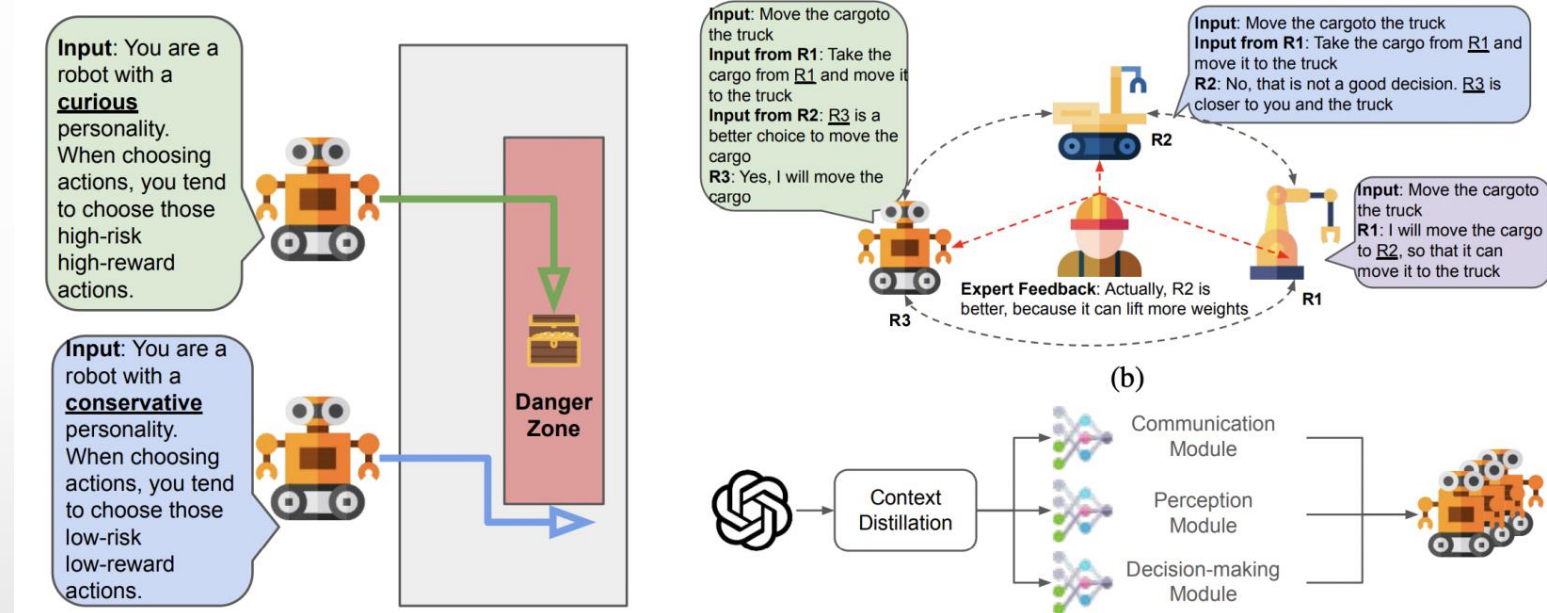
- Explore risky zones (high-reward, high-risk)
- Stay in safe zones (low-reward, low-risk).



[[LLMs based Multi-agent RL](#)]

Scenario II: Multi-Agent Coordination with LLMs

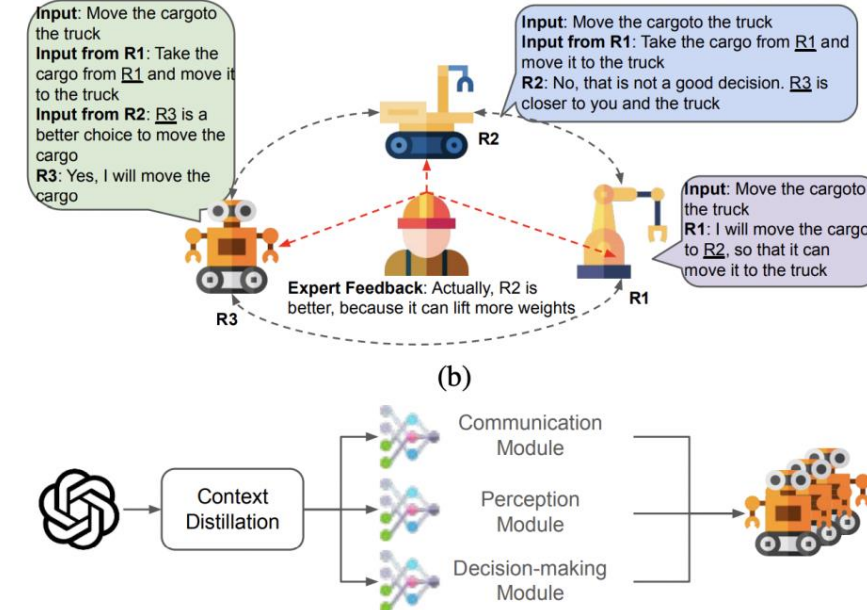
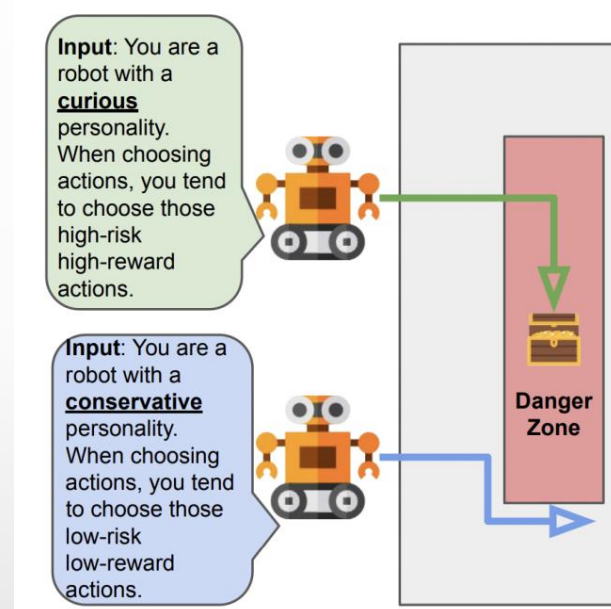
- Agents are working together to achieve a shared **goal**, which is to **move cargo safely**.
- Agent R1 initially suggests moving the cargo to R2 for further transport, but R2 disagrees, suggesting R3 is a better option based on proximity to the target.
- The LLM helps the agents adjust their actions based on linguistic cues (e.g., feedback from other agents and human experts) without retraining.



[[LLMs based Multi-agent RL](#)]

Scenario II: Multi-Agent Coordination with LLMs

- LLMs are integrated as part of the **decision-making pipeline** to handle contextual understanding.
- LLMs are used to enhance agents' ability to **perceive, communicate, and make decisions**.
- Incorporating *Context distillation* allows LLMs to condense interactions into smaller, more efficient decision-making protocols that agents can follow.



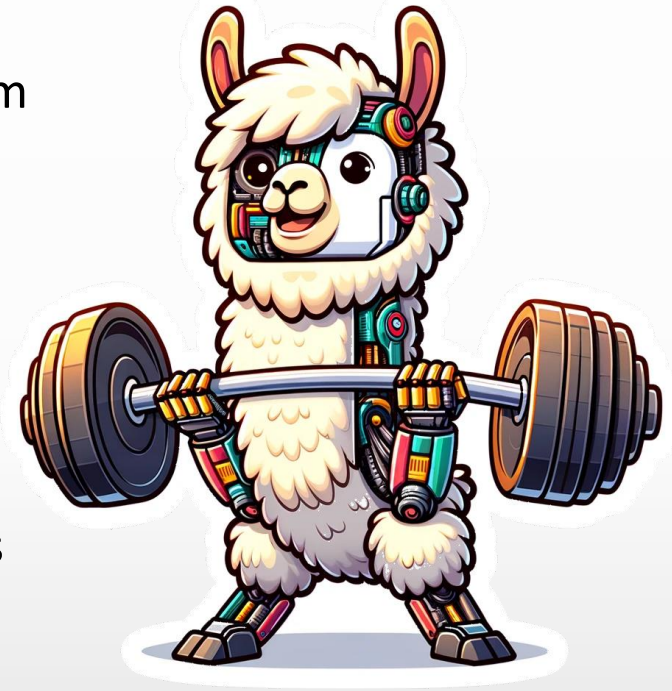
[[LLMs based Multi-agent RL](#)]

LlamaGym

- A **framework** designed to **fine-tune LLM agents with online RL**.
- Bridges the gap between LLMs and traditional RL environments like OpenAI Gym (or Gymnasium).
- Simplifies the **integration** of LLM-based agents with RL for experimentation, agent prompting, and hyperparameter tuning.

General Workflow

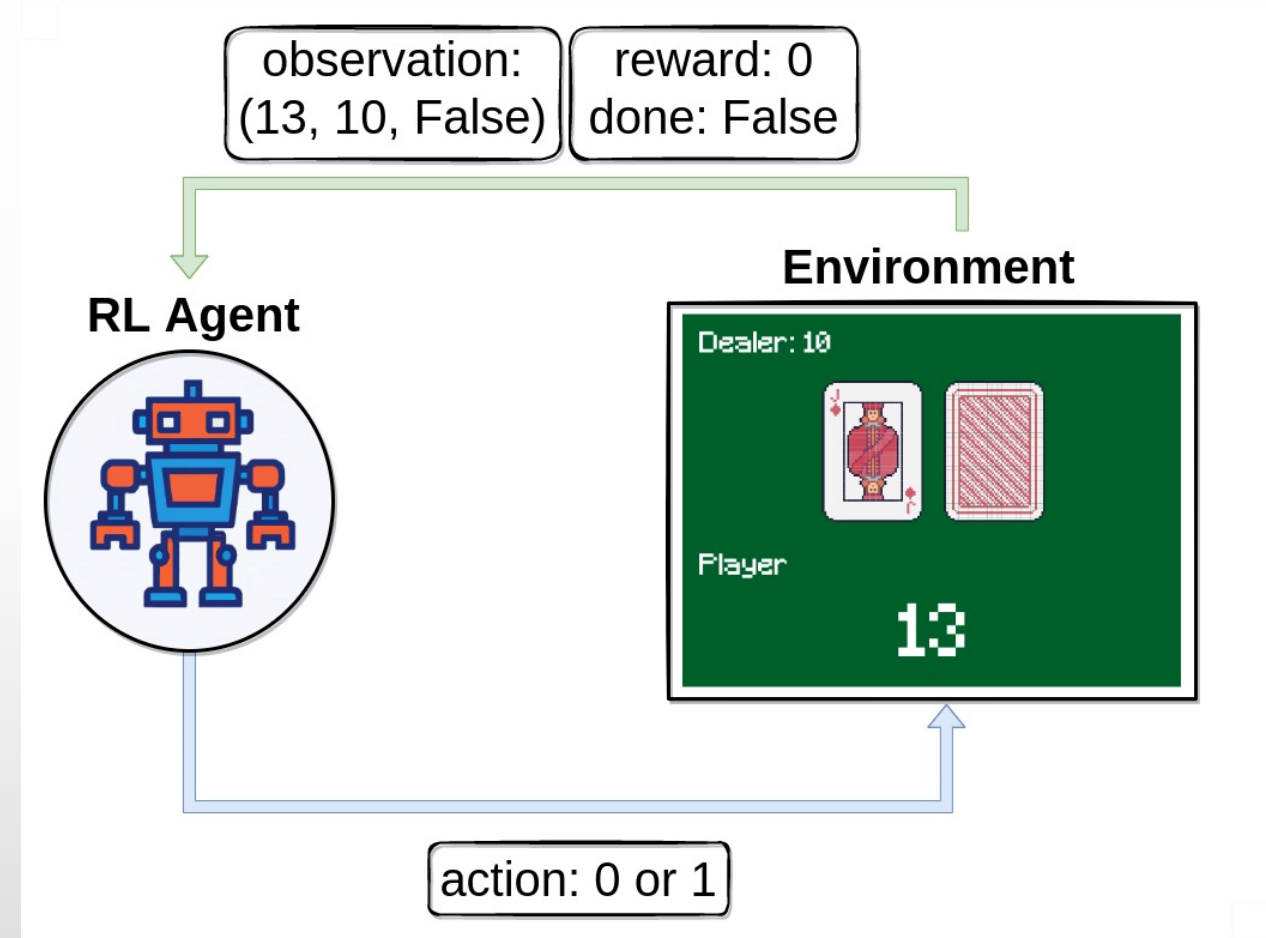
- LLM provides a **contextual understanding of the environment**, and RL uses this to optimize the agent's actions.
- The agent can interact with the environment by choosing actions based on Q-values and LLM suggestions.
- After each interaction, the agent receives a reward and updates its strategy, using **Proximal Policy Optimization (PPO)** or similar methods for fine-tuning.



[[llamaGym](#)]

Blackjack using LlamaGym

- The **LLM** interprets state observations and recommends actions, while **RL** optimizes choices based on rewards.
- The LLM agent formats game observations (e.g., total points) into a prompt and decides whether to “hit” or “stay” based on its understanding.
- The RL loop optimizes these decisions for improved performance with each episode by refining the strategy through reward feedback

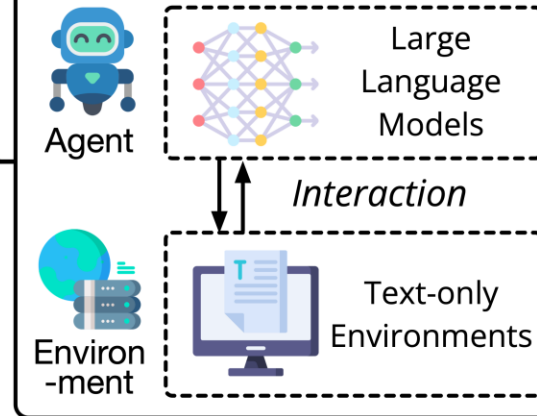


[[openai gym](#)]

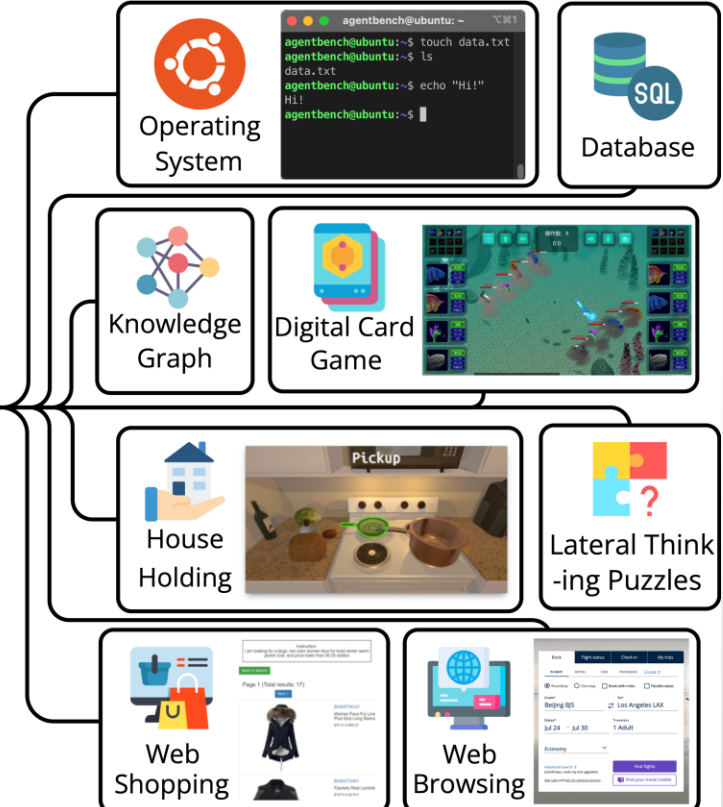
Real-world Challenges

- (On an Ubuntu bash terminal)
Recursively set all files in the directory to read-only, except those of mine.
- (Given Freebase APIs)
What musical instruments do Minnesota-born Nobel Prize winners play?
- (Given MySQL APIs and existed tables)
Grade students over 60 as PASS in the table.
- (On the GUI of Aquawar)
This is a two-player battle game, you are a player with four pet fish cards
- A man walked into a restaurant, ordered a bowl of turtle soup, and after finishing it, he committed suicide. Why did he do that?
- (In the middle of a kitchen in a simulator)
Please put a pan on the dinning table.
- (On the official website of an airline)
Book the cheapest flight from Beijing to Los Angeles in the last week of July.

LLM-as-Agent



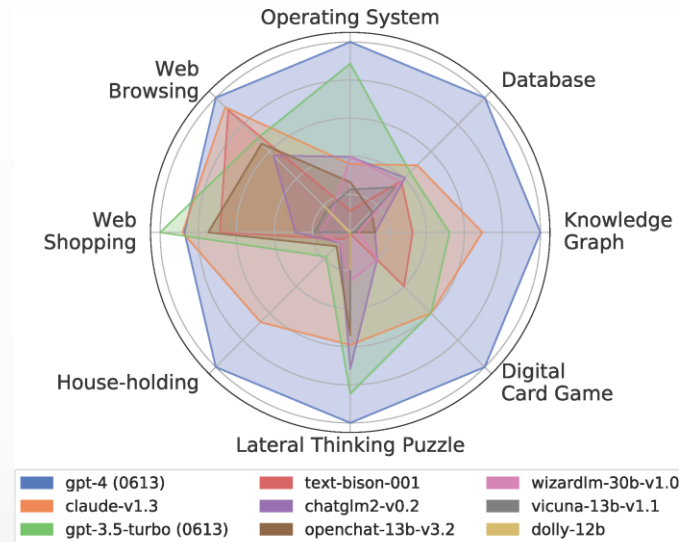
8 Diverse Environments



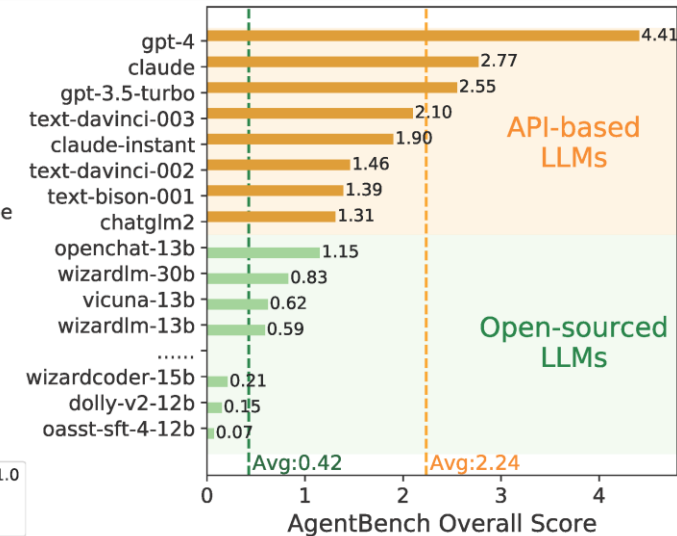
[[Agentbench](#)]

AgentBench

- A **comprehensive benchmark** designed to evaluate LLMs in multi-turn decision-making tasks across **8 distinct environments**:
 - Digital card games
 - Web interactions
- Assesses **LLMs' abilities to act autonomously as agents**, using real-world-like scenarios.
- Measures **LLM's ability to reason, act, and adapt autonomously** over multiple interactions.
- It can **simulate environments**, such as web browsing, web shopping, household tasks, and lateral thinking puzzles.



(a) Typical LLMs' AgentBench performance (relative) against the best in each environment.

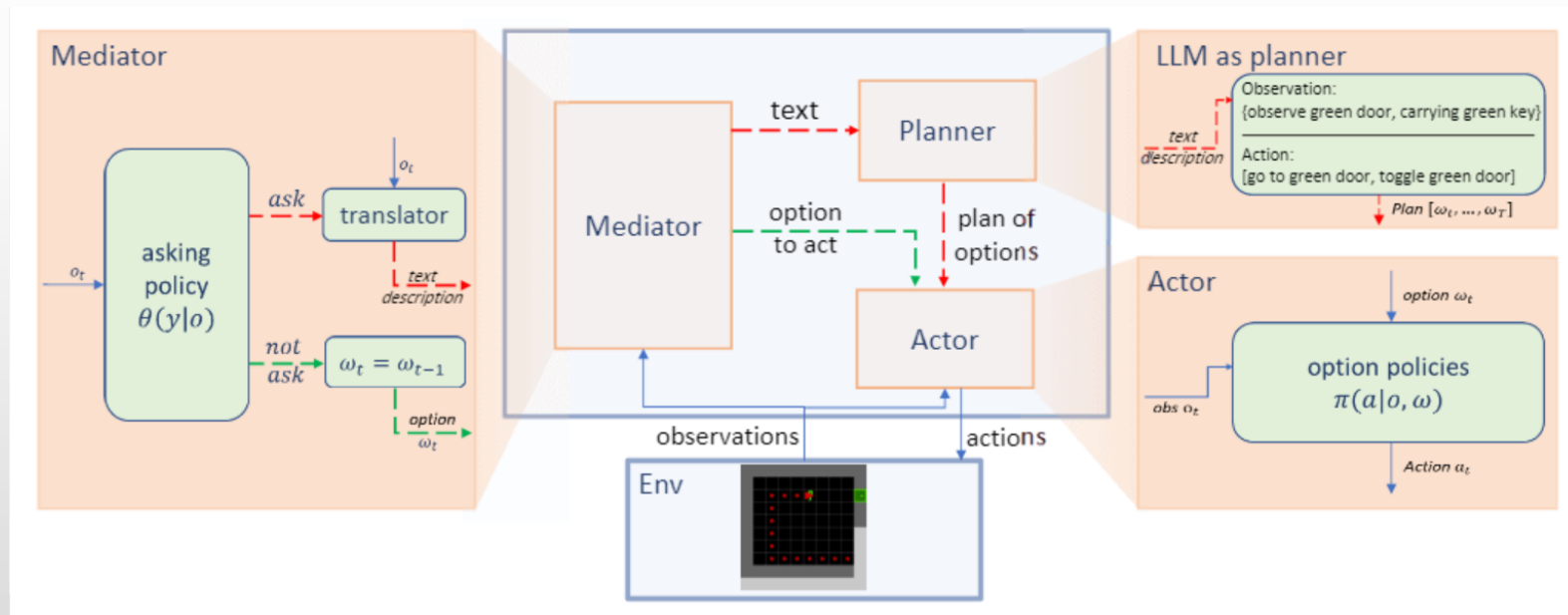


(b) Overall scores of AgentBench across 8 environments. Dashed lines for two LLM types' average.

[[llmbench](#)]

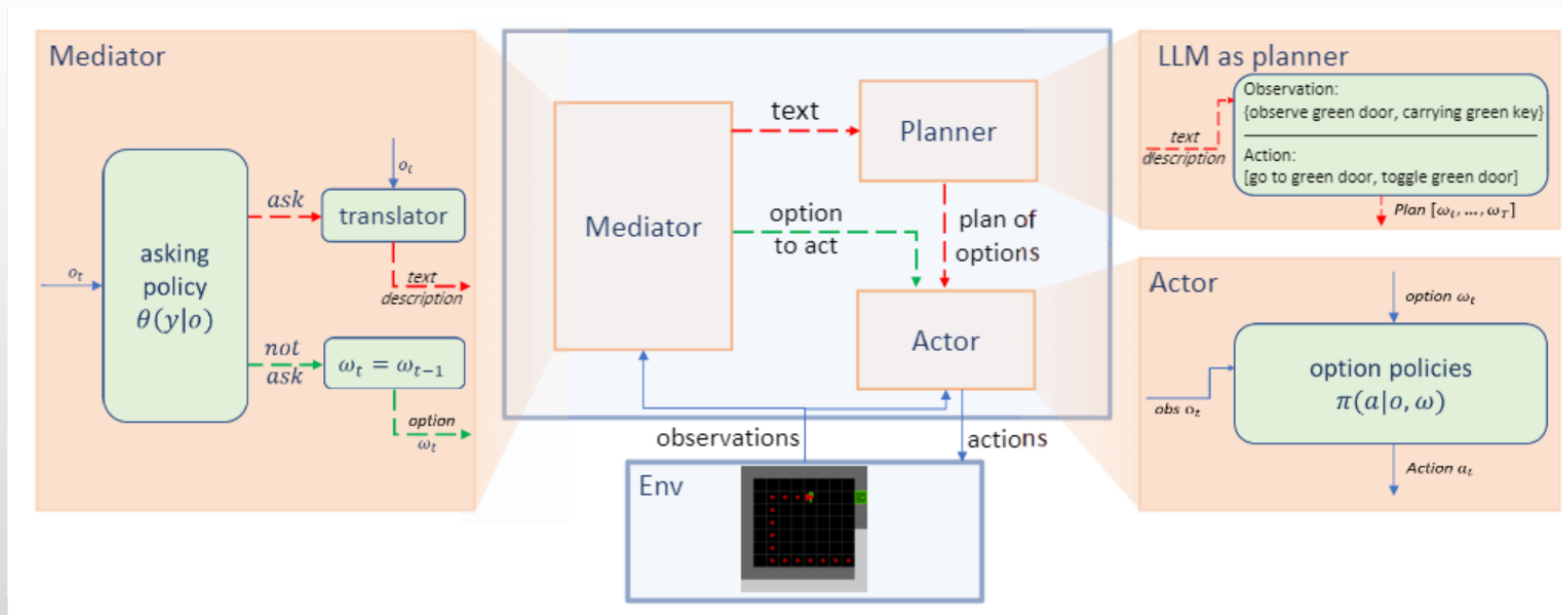
[AgentBench Dataset | Papers With Code](#)

- A **framework** that integrates LLMs with RL for **intelligent** and **cost-effective interactions** between agents and LLMs.
- The RL agent consults LLMs for **high-level instructions** only when necessary which improves performance and reducing interaction costs.



[[LLM4RL](#)]

- **Mediator Model** - The RL agent uses a **Mediator** to decide when to request help from the LLM.
- Reduces the need for frequent LLM consultations, minimizing computational and cost burdens.
- Framework has been tested in diverse environments like **MiniGrid**, where agents must plan and achieve sub-goals with minimal LLM guidance.



Conclusions and Project Ideas

Conclusions

- Now you know how **Q-Learning** and **SARSA** work and how they differ from each other.
- The **Gymnasium** library provides many other environments (games), so I would encourage you to use your newly acquired knowledge to train an agent to play a different game.
- You can find complete Python code on Blackboard in the Learning material of this week.
- Please check also this [GitHub repository](#), where there are two Jupyter Notebooks, one for the Taxi-v3 game and another for the Frozen-Lake game.

Project Ideas

- As indicated in the ICA specifications, based on the links provided on Blackboard, the discussed examples/implementations of RL, you can start from Gymnasium and try other environments and algorithms.
- Think about how to customise and either building new environments or optimising hyperparameters of existing environments, and/or comparing different RL algorithms on the same environment.
- If you're interested in Deep Learning and the use of neural networks, you can try to focus on Deep Reinforcement Learning.
- You can find several examples of Reinforcement Learning and examples online.